

PPE GUARDIAN · TECHNICAL OVERVIEW FOR THE CLIENT'S UI DEVELOPERS

Dashboard & Backend at a Glance

What the current dashboard contains, what each screen does, and how a new user interface connects to the backend. Version 1.0 · 30 September 2026 · Prepared by Nayatel AI

1. How we will work together

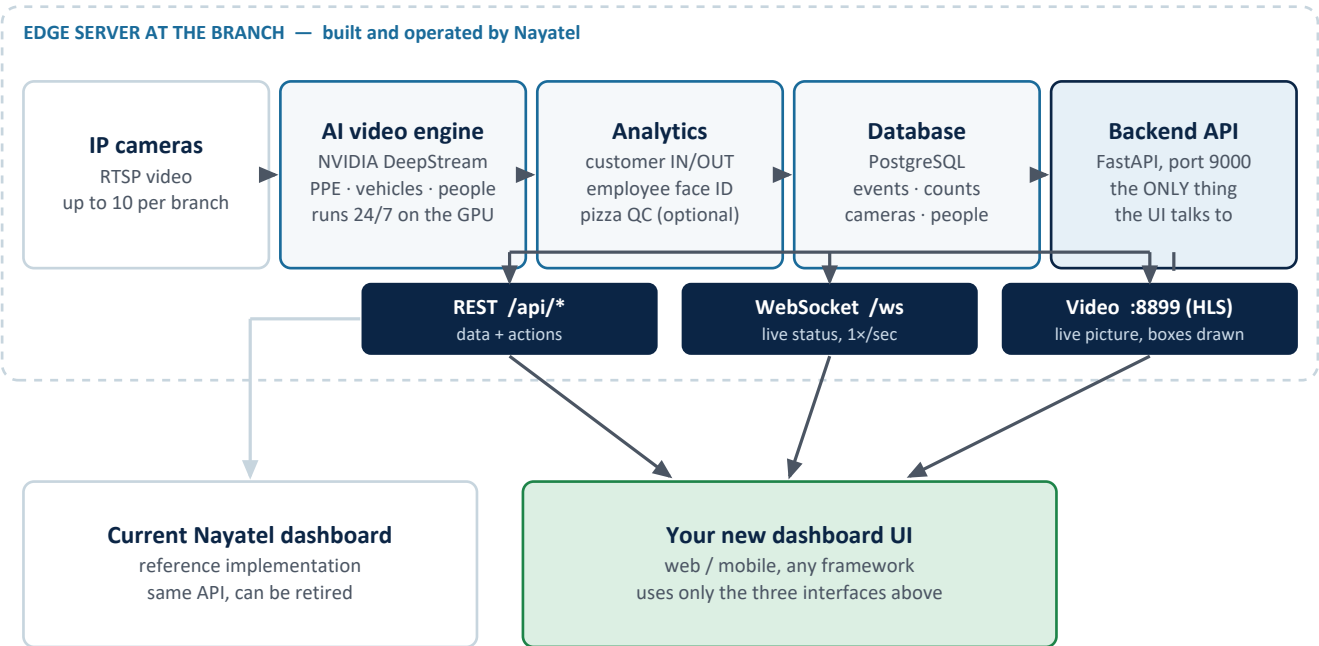
Your team designs the user interface and shares the template (Figma or HTML/CSS). Nayatel connects every screen of that template to the existing backend. All data and actions the dashboard shows today are already available through the backend's interfaces, so no screen in your design needs new AI work — only wiring.

#	Step	Who / what
1	Read this document	Your UI team — screens, what they do, which data exists
2	Design the template	Your UI team — any look and any framework (React, Vue, Angular, mobile)
3	Review against the API	Joint — confirm every widget in the design has a data source (Section 5)
4	Integrate	Nayatel — connect the template to REST, WebSocket and video
5	Acceptance	Joint — each screen checked against the live system at the branch

The one principle to design around

The backend does all the AI work continuously, whether or not anyone has the dashboard open. The user interface only **shows** results and **sends** settings. It never processes video itself. Live video is played for **one selected camera at a time**: the dashboard lists cameras as tiles or icons, and choosing one opens its video.

2. System at a glance

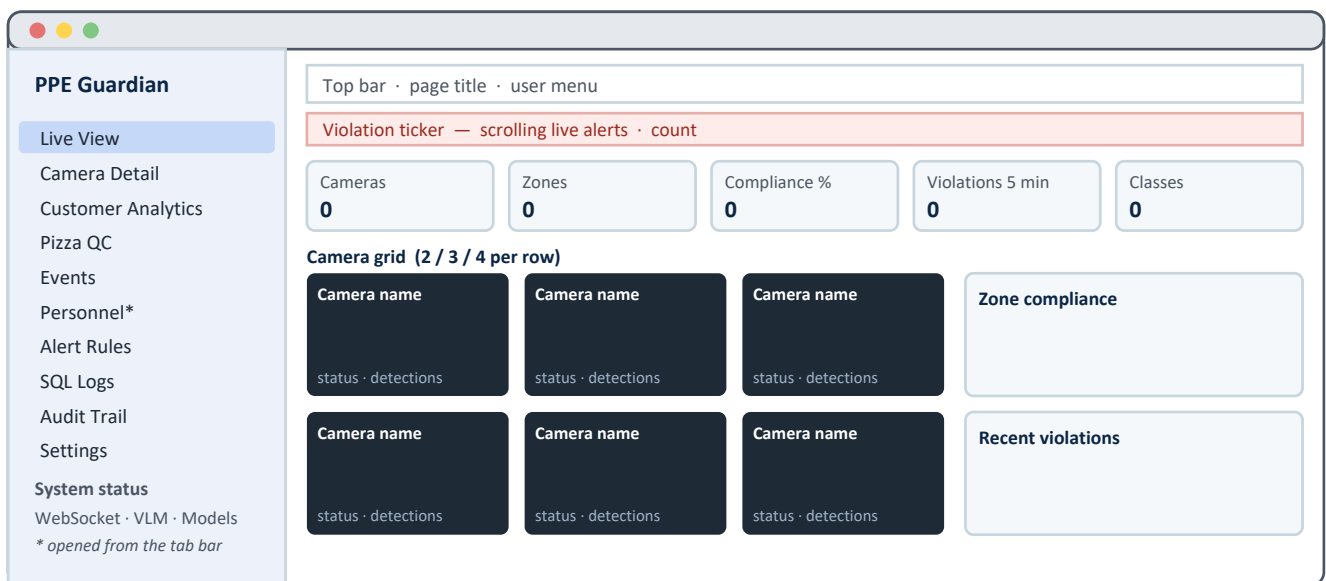


The backend exposes exactly three interfaces. A new UI uses only these; it never talks to the cameras, the AI engine or the database directly.

Interface	Address	Used for
REST API	http://<server>:9000/api/...	Reading data (cameras, events, counts, reports) and every action (save zones, change models, review events). JSON in, JSON out.
WebSocket	ws://<server>:9000/ws	Live status pushed about once per second: every camera's health, current detections and active violations. Drives the ticker, tiles and badges.
Live video (HLS)	http://<server>:8899/<camera_id>/index.m3u8	The annotated live picture for the selected camera, with detection boxes already drawn on the server. Plays in any browser via hls.js (native on Safari/iOS).

3. The current dashboard

A single-page web application: a sidebar for navigation, a top bar with a live violation ticker, and a main content area. The sketch below shows the layout of the main screen; your design is free to change it.



Elements present on every screen

- **Sidebar navigation** to the ten screens listed in Section 4, and a **System status** panel (WebSocket connected, AI summary on/off, models loaded, classes).
- **Violation ticker** — a scrolling banner of current violations with a count. Fed by the WebSocket's live alerts list.
- **Add Camera** and **Refresh** buttons, and the page title with a one-line description.

4. Screen inventory

Every screen in the current dashboard, what it is for, and our recommendation for the new UI. **Core** = needed by branch staff and managers. **Scope-dependent** = include if that capability is purchased. **Admin** = configuration screens, can be simpler or restricted to administrators. **Internal** = engineering tool, do not include.

Screen	Purpose	For the new UI
Live View	Overview of all cameras: headline figures, camera tiles, zone compliance, recent violations	Core
Camera Detail	One camera in depth: live video, health, compliance trend, AI models on it, zones, recording policy, its events	Core
Customer Analytics	Customers entered, left, still inside — by hour, by day, last 7 days — with a short video clip of every count	Core
Events	Log of every violation with image; filter, review (acknowledge, dismiss, resolve) and export	Core
Personnel	Employee face enrolment, list of known and unrecognised people, daily per-person violation report	Scope-dependent (employee recognition)
Pizza QC	Latest pizza with topping counts and recipe verdict; today's summary; recent pizzas	Scope-dependent (pizza QC)
Settings	WhatsApp alert set-up and recipients, AI text summaries on/off, detection sensitivity	Admin
Alert Rules	Webhook rules that forward violations to other systems	Admin
Audit Trail	Every change made through the system: who changed what and when	Admin
SQL Logs	Raw database table browser used by engineers	Internal — do not include

5. What each screen does

For each screen: what it shows, what the user can do, and where the data comes from. Endpoint paths are relative to `http://<server>:9000`.

5.1 Live View

The home screen. Gives a manager the state of the whole branch at a glance.

Shows	Five headline cards (cameras, zones, overall compliance %, violations in the last 5 minutes, detectable classes) · a grid of camera tiles, each with name, status and current detections · zone compliance list · recent violations list.
User actions	Change grid density (2/3/4 per row) · show or hide AI analysis on tiles · open a camera · add a camera.
Data from	GET <code>/api/dashboard</code> (headline cards, recent violations) · WebSocket <code>/ws</code> (tiles, live status) · GET <code>/api/cameras/{id}/frame.jpg</code> (still image for a tile)

Design note: tiles should show a still image or status, not live video. Video opens for the one camera the user selects.

5.2 Camera Detail

Everything about one camera.

Shows	Live video with detection boxes · health (frames per second, AI time, stream health, reconnects) · compliance and violation count · hourly compliance trend · AI text summary of the latest violation · which AI models run on the camera · zones drawn on the picture · recording policy · the camera's recent events.
User actions	Pick camera · switch AI models on/off for this camera (PPE, vehicles, people counting, fire, pizza) · draw and edit zones and their rules · edit recording policy · remove camera.
Data from	HLS :8899/{id}/index.m3u8 · WebSocket /ws · GET/PUT /api/cameras/{id}/pipelines · GET/PUT /api/zones/{id} · GET/PUT /api/cameras/{id}/recording-policy · GET /api/cameras/{id}/compliance-trend · DELETE /api/cameras/{id}

When several models run on one camera, the first one in the list draws its boxes on the video; the others still detect and record events.

5.3 Customer Analytics

How many customers came in and went out, and when.

Shows	For a chosen date: totals entered / left / still inside / peak · hour-by-hour bar chart and table · last 7 days · list of every count with a ±10-second video clip for verification.
User actions	Choose date · switch view (by hour, hourly detail, last 7 days) · play the clip behind any count.
Data from	GET /api/analytics/people?day=YYYY-MM-DD · GET /api/analytics/people/range?days=7 · GET /api/analytics/people/clips · GET /api/analytics/people/clips/{file} (MP4) · GET /api/analytics/people/live

5.4 Events

The violation log — the record managers act on.

Shows	Paginated table: time, camera, zone, violation type, severity, status, person (when recognised), snapshot image.
User actions	Filter by severity, status, camera and date · open an event · set status (Open, Acknowledged, Dismissed, False positive, Resolved) with assignee and note · export CSV.
Data from	GET /api/events?limit&offset&camera_id&severity&status&date_from&date_to · GET /api/events/{id} · PATCH /api/events/{id}

5.5 Personnel (employee recognition)

Who the system knows, and each employee's record.

Shows	Face engine status · enrolled employees (name, employee code, role, last seen, today's violations) · auto-discovered unrecognised people · daily violation report per person.
User actions	Enrol an employee with 5–10 photos · edit or delete a person · delete all unrecognised · run the daily report and download CSV.
Data from	GET/POST /api/persons · POST /api/persons/enroll · PATCH/DELETE /api/persons/{id} · GET /api/persons/{id}/report · GET /api/persons-report/daily?date= · GET /api/face-engine/stats

Employee entry/exit times will be added to this screen as part of the employee recognition deliverable.

5.6 Pizza QC (optional capability)

Quality check of every pizza on the cut table.

Shows	Latest pizza image with toppings boxed, topping counts, recipe verdict (PASS / REVIEW) and reason · today's totals · strip of recent pizzas.
User actions	Select any recent pizza to view it · return to latest. The screen refreshes itself every 5 seconds.
Data from	GET /api/analytics/pizza/latest · GET /api/analytics/pizza/history?day=&limit= · GET /api/analytics/pizza/image/{day}/{file}

5.7 Settings, Alert Rules, Audit Trail (administration)

Configuration screens, normally used by an administrator only.

Shows	WhatsApp alerts (on/off, minimum severity, cooldown, with or without image) and recipients · AI text summaries on/off · per-class detection sensitivity · webhook alert rules · audit trail of all changes.
User actions	Save WhatsApp settings · add/remove recipients · send a test message · add/delete webhook rules · adjust sensitivity.
Data from	GET/PUT /api/settings/whatsapp · .../whatsapp/recipients · POST .../whatsapp/test · GET/POST/DELETE /api/alerts · GET /api/audit · GET/PUT /api/settings/inference/class-thresholds · GET /api/vlm · POST /api/vlm/toggle

Engineering-level tuning panels in the current Settings screen are used by Nayatel and do not need to appear in the new UI.

6. The data your screens will display

The main objects returned by the backend, with the fields a designer needs. Field names are as they appear in the JSON.

Live status message (WebSocket, about once per second)

Field	Meaning
type	Always status_update
timestamp	Server time, UTC, ISO-8601
cameras.{camera_id}	One entry per camera: name, running, fps, stream_healthy, compliance (%), violation_count, active_detections, active_violations_count, detections[], recent_events[], zones[], ds_pipelines (AI models on the camera)
live_alerts[]	Current violations across all cameras (max 20): camera, violation type, zone, severity, confidence. Feeds the ticker.

Detection (inside a camera's detections[])

Field	Meaning
class_name	What was detected, e.g. no_gloves, person, car
confidence	0–1
bbox	Box [x1, y1, x2, y2] in pixels of the camera frame (only needed if the UI draws its own overlays)
severity	high / medium / low
zone_name, zone_violation	Zone the detection is in, and whether it breaks that zone's rules

Event (violation record)

Field	Meaning
id, timestamp	Unique id; time in UTC
camera_id, zone_name	Where it happened
class_name, severity, confidence	What was detected and how serious
status, assignee, corrective_note	Review workflow: open, acknowledged, dismissed, false_positive, resolved
Snapshot	Image of the moment, served under /media/...

Customer counts (/api/analytics/people)

Field	Meaning
day	Date of the report, YYYY-MM-DD
totals	entered, left, still_inside, peak_seen
hours[]	24 entries: hour, entered, left, peak_seen
live	Current state of the door camera

Pizza reading (/api/analytics/pizza/latest)

Field	Meaning
at	Time of the reading
type	Recognised pizza type, e.g. Pepperoni
counts	Toppings counted, e.g. {"pepperoni": 18, "black_olive": 6}
verdict, issues	PASS or REVIEW, with the reasons
image	Path of the annotated image, fetched from /api/analytics/pizza/image/...

7. Integration notes for the UI team

Topic	What to design for
Live video	Play one camera at a time, on user selection (HLS via hls.js, native on Safari/iOS). Do not auto-play a wall of videos: the server encodes video only for cameras being watched, and the design should assume that.
Refresh	Use the WebSocket for anything live (tiles, ticker, health). Reports and lists are fetched on demand; the Pizza QC screen refreshes every 5 seconds.
Time	Timestamps are UTC (ISO-8601). Show them in branch local time (Saudi Arabia, UTC+3).
Lists	Long lists are paginated with limit and offset.
Optional features	If a capability is not enabled at a branch (e.g. customer counting, employee recognition), its endpoints return HTTP 503. Show a clear "not enabled" state rather than an error.
Sign-in	The API currently runs inside the branch network without sign-in. A login step with a token on every request will be added for the new UI — please include a login screen in the design.
Images & clips	Snapshots, pizza images and count clips are ordinary URLs returned by the API; display them directly.
Reference	The current dashboard is one HTML file that uses exactly these interfaces. Nayatel can share sample JSON responses for each screen to help your designers work with realistic data.

In one sentence

Design any interface you like around the ten screens above; every figure, list, image and video it needs is already served by the backend's REST API, WebSocket and HLS video, and Nayatel will connect your template to them.